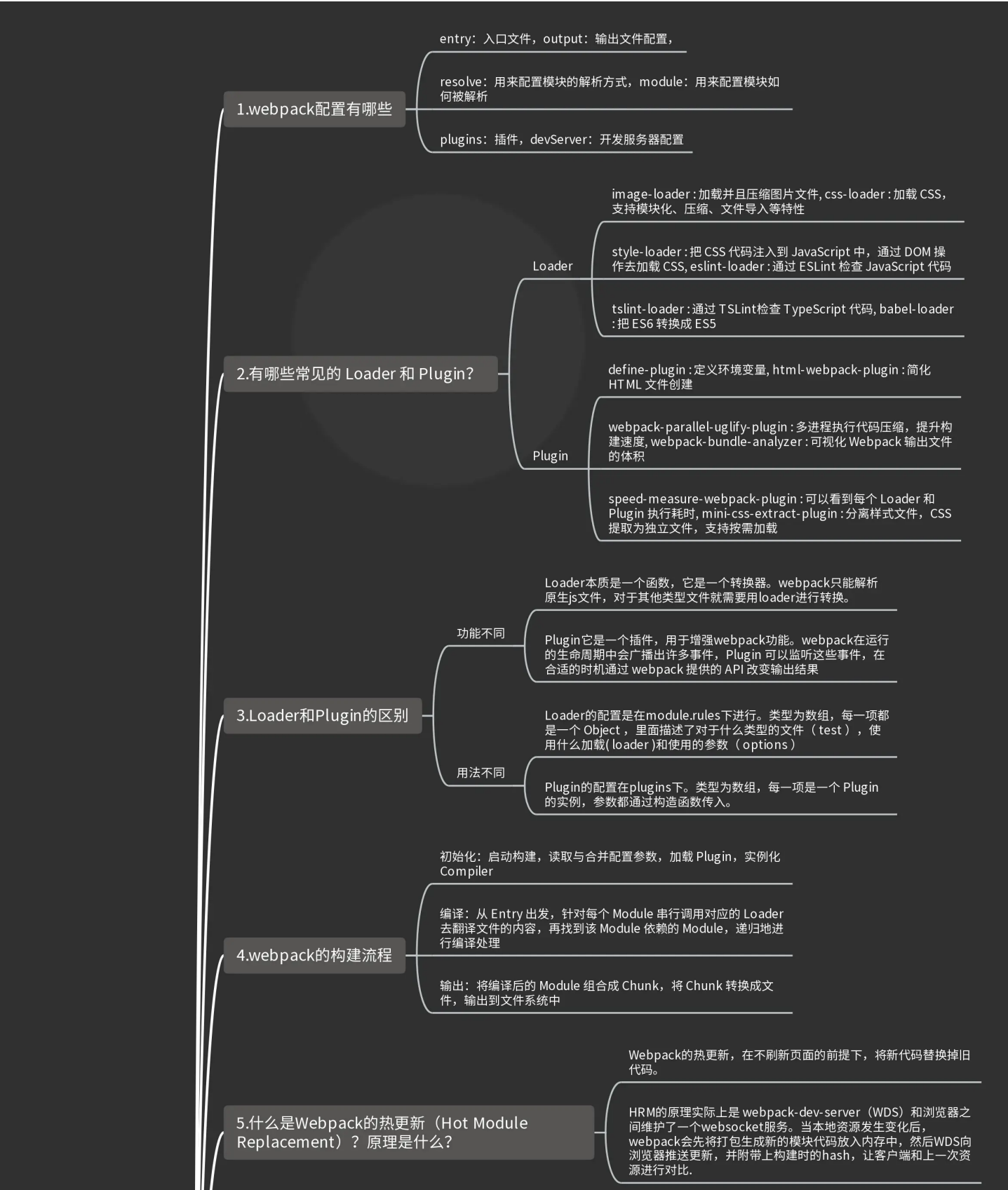


2024前端高频面试题之-- 前端工程化篇

【导读】 本文总结了前端面试中经常问到的前端工程化高频面试题，仅供参考。

下图为思维导图：



前端工程化

6.什么是Code Splitting

Code Splitting 代码分割，是一种优化技术。它允许将一个大的 chunk 拆分成多个小的 chunk，从而实现按需加载，减少初始加载时间，并提高应用程序的性能

在Webpack中通过`optimization.splitChunks`配置项来开启代码分割

7.Webpack的Source Map是什么？如何配置生成Source Map？

Source Map是一种文件，它建立了构建后的代码与原始源代码之间的映射关系。通常在开发阶段开启，用来调试代码，帮助找到代码问题所在

在Webpack配置文件中的`devtool`选项中指定`devtool: 'source-map'`来开启

8.Webpack的Tree Shaking原理

Tree Shaking 也叫摇树优化，是一种通过移除多于代码，从而减小最终生成的代码体积，生产环境默认开启。

ES6 模块系统: Tree Shaking的基础是ES6模块系统

静态分析: 在Webpack构建过程中，Webpack会通过静态分析依赖图

标记未使用代码: 在分析模块依赖时，Webpack会标记每个变量、函数、类和导入，以确定它们是否被实际使用

删除未使用代码: 在代码标记为未使用后，Webpack会在最终的代码生成阶段，通过工具（如UglifyJS等）删除这些未使用的代码

9.如何提高webpack的打包速度

利用缓存: 利用Webpack的持久缓存功能，避免重复构建没有变化的代码

使用多进程/多线程构建: 使用`thread-loader`、`happypack`等插件可以将构建过程分解为多个进程或线程

使用`DllPlugin`和`HardSourceWebpackPlugin`: `DllPlugin`可以将第三方库预先打包成单独的文件，减少构建时间。`HardSourceWebpackPlugin`可以缓存中间文件，加速后续构建过程

使用Tree Shaking: 配置Webpack的Tree Shaking机制，去除未使用的代码，减小生成的文件体积

移除不必要的插件: 移除不必要的插件和配置，避免不必要的复杂性和性能开销

10.如何减少打包后的代码体积

代码分割 (Code Splitting): 将应用程序的代码划分为多个代码块，按需加载

Tree Shaking: 配置Webpack的Tree Shaking机制，去除未使用的代码

压缩代码: 使用工具如UglifyJS或Terser来压缩JavaScript代码

使用生产模式: 在Webpack中使用生产模式，通过设置`mode: 'production'`来启用优化

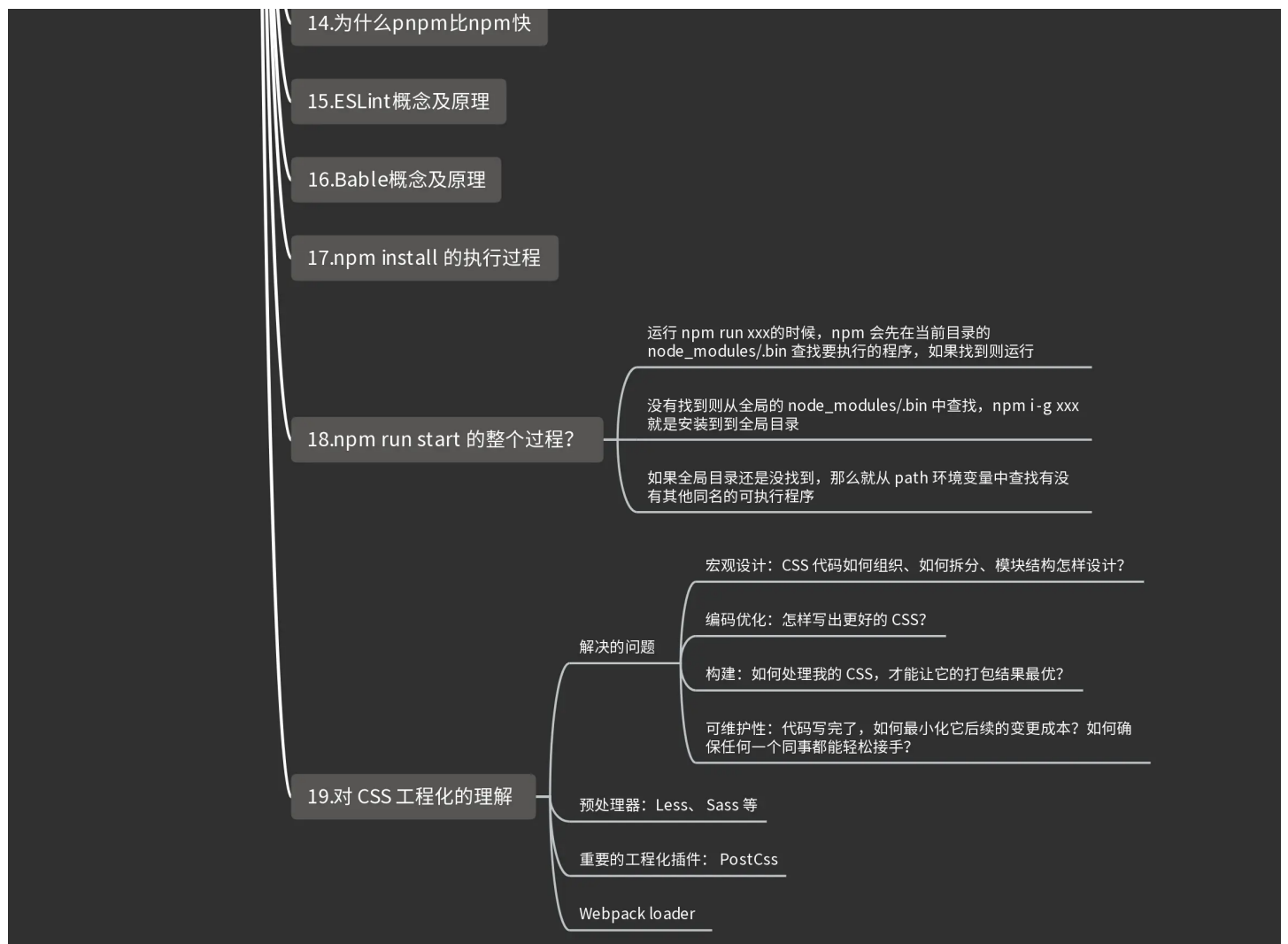
使用压缩工具: 使用现代的压缩工具，如Brotli和Gzip，来对静态资源进行压缩

利用CDN加速: 将项目中引用的静态资源路径修改为CDN上的路径，减少图片、字体等静态资源等打包

11.vite比webpack快在哪里

12.说一下你对Monorepo的理解

13.你在项目是怎么做Monorepo?



1.webpack配置有哪些？

Webpack的配置主要包括以下几个部分：

- entry。指定Webpack打包的入口文件，可以是单个或多个JavaScript文件。这个配置决定了Webpack从哪个模块开始生成依赖关系图。1234
- output。设置Webpack打包后输出的目录和文件名称，包括path、filename和publicPath等。235
- module。配置了不同的loaders来处理不同的模块，例如，对于CSS文件，可以使用css-loader和style-loader。2345
- resolve[🔗]。设置Webpack如何解析模块依赖，包括别名、扩展名等。
- plugins[🔗]。使用不同的插件可以增强Webpack的功能，例如，使用html-webpack-plugin可以将打包后的js文件自动引用到HTML文件中。
- devServer[🔗]。提供了一个简单的web服务器和实时重载功能，可以通过devServer.contentBase、devServer.port、devServer.proxy等进行配置。
- optimization[🔗]。可以使用optimization.splitChunks和optimization.runtimeChunk配置代码拆分和运行时代码提取等优化策略。

- `externals`。用于配置排除打包的模块，例如，可以将jQuery作为外置扩展，避免将其打包到应用程序中。
- `devtool`。配置source-map类型。
- `context`。webpack使用的根目录，string类型必须是绝对路径。
- `target`。指定Webpack编译的目标环境。
- `performance`。输出文件的性能检查配置。
- `noParse`。不用解析和处理的模块。
- `stats`。控制台输出日志控制。

2. 有哪些常见的 Loader 和 Plugin?

Loader:

- `babel-loader`: 将ES6+的代码转换成ES5的代码。
- `css-loader`: 解析CSS文件，并处理CSS中的依赖关系。
- `style-loader`: 将CSS代码注入到HTML文档中。
- `file-loader`: 解析文件路径，将文件赋值到输出目录，并返回文件路径。
- `url-loader`: 类似于file-loader，但是可以将小于指定大小的文件转成base64编码的Data URL格式。
- `sass-loader`: 将Sass文件编译成CSS文件。
- `less-loader`: 将Less文件编译成CSS文件。
- `postcss-loader`: 自动添加CSS前缀，优化CSS代码等。
- `vue-loader`: 将Vue单文件组件编译成JavaScript代码。

Plugin:

- `HtmlWebpackPlugin`: 生成HTML文件，并自动将打包后的JavaScript和CSS文件引入到HTML文件中。
- `CleanWebpackPlugin`: 清除输出目录。
- `ExtractTextWebpackPlugin`: 将CSS代码提取到单独的CSS文件中。
- `DefinePlugin`: 定义全局变量。
- `UglifyJsWebpackPlugin`: 压缩JavaScript代码。
- `HotModuleReplacementPlugin`: 热模块替换，用于在开发环境下实现热更新。
- `MiniCssExtractPlugin`: 与ExtractTextWebpackPlugin类似，将CSS代码提取到单独的CSS文件中。
- `BundleAnalyzerPlugin`: 分析打包后的文件大小和依赖关系。

3.Loader和Plugin的区别

功能不同：

Loader本质是一个函数，它是一个转换器。webpack只能解析原生js文件，对于其他类型文件就需要loader进行转换。

Plugin它是一个插件，用于增强webpack功能。webpack在运行的生命周期中会广播出许多事件，Plugin可以监听这些事件，在合适的时机通过webpack提供的API改变输出结果。

用法不同：

Loader的配置是在module.rules下进行。类型为数组，每一项都是一个Object，里面描述了对于什么类型的文件（test），使用什么加载(loader)和使用的参数(options)。

Plugin的配置在plugins下。类型为数组，每一项是一个Plugin的实例，参数都通过构造函数传入。

4.webpack的构建流程

Webpack的构建流程主要包括以下几个步骤：

1. 初始化参数。解析Webpack配置参数，合并Shell传入和webpack.config.js文件配置参数，形成最终的配置结果。
2. 开始编译。使用上一次得到的参数初始化compiler对象，注册所有配置的插件，插件监听Webpack构建生命周期的事件节点，做出相应的反应，执行对象的run方法开始执行编译。
3. 确定入口。从配置的entry入口，开始解析文件构建AST语法树，找出依赖，递归下去。
4. 编译模块。递归中根据文件类型和loader配置，调用所有配置的loader对文件进行转换，再找出该模块依赖的模块，再递归本步骤直到所有入口依赖的文件都经过了本步骤的处理。
5. 完成模块编译。在经过第四步使用Loader翻译完所有模块后，得到了每个模块被翻译后的最终内容以及它们之间的依赖关系。
6. 输出资源。根据入口和模块之间的依赖关系，组装成一个个包含多个模块的Chunk，再把每个Chunk转换成单独的文件加入到输出列表，这步是可以修改输出内容的最后机会。
7. 输出完成。在确定好输出内容后，根据配置确定输出的路径和文件名，把文件内容写入到文件系统。

这个流程是一个串行的过程，Webpack的运行流程是一个串行的过程，它的工作流程就是将各个插件串联起来。在运行过程中会广播事件，插件只需要监听它所关心的事件，就能加入到这条Webpack机制中，去改变Webpack的运作，使得整个系统扩展性良好。

5.什么是Webpack的热更新（Hot Module Replacement）？原理是什么？

Webpack的热更新，在不刷新页面的前提下，将新代码替换掉旧代码。

HRM的原理实际上是webpack-dev-server（WDS）和浏览器之间维护了一个websocket服务。当本地资源发生变化后，webpack会先将打包生成新的模块代码放入内存中，然后WDS向浏览器推送更

新，并附上构建时的hash，让客户端和上一次资源进行对比。

6.什么是Code Splitting

Code Splitting代码分割，是一种优化技术。它允许将一个大的chunk拆分成多个小的chunk，从而实现按需加载，减少初始加载时间，并提高应用程序的性能。

在Webpack中通过optimization.splitChunks配置项来开启代码分割

7.Webpack的Source Map是什么？如何配置生成Source Map？

Source Map是一种文件，它建立了构建后的代码与原始源代码之间的映射关系。通常在开发阶段开启，用来调试代码，帮助找到代码问题所在。

在Webpack配置文件中的devtool选项中指定devtool: 'source-map'来开启

8.Webpack的Tree Shaking原理

Webpack的Tree Shaking是一个利用ES6模块静态结构特性来去除生产环境下不必要代码的优化过程。其工作原理在于：

1. 当Webpack分析代码时，它会标记出所有的import语句和export语句。
2. 然后，当Webpack确定某个模块没有被导入时，它会在生成的bundle中排除这个模块的代码。
3. 同时，Webpack还会进行递归的标记清理，以确保所有未使用的依赖项都不会出现在最终的bundle中。

为了启用Tree Shaking，需要在webpack配置文件中添加如下设置：

```
1  yaml
2  复制代码
3  javascriptmodule.exports = { // ... optimization: {    usedExports: true,
    concatenateModules: true,    minimize: true, }, // ...};
```

确保你使用的是ES6模块语法（即import和export），因为只有这样才能让Tree Shaking发挥作用。

9.如何提高webpack的打包速度

- 利用缓存：利用Webpack的持久缓存功能，避免重复构建没有变化的代码
- 使用多进程/多线程构建：使用thread-loader、happypack等插件可以将构建过程分解为多个进程或线程
- 使用DllPlugin和HardSourceWebpackPlugin：DllPlugin可以将第三方库预先打包成单独的文件，减少构建时间。HardSourceWebpackPlugin可以缓存中间文件，加速后续构建过程
- 使用Tree Shaking: 配置Webpack的Tree Shaking机制，去除未使用的代码，减小生成的文件体积
- 移除不必要的插件: 移除不必要的插件和配置，避免不必要的复杂性和性能开销

10.如何减少打包后的代码体积

- 代码分割（Code Splitting）：将应用程序的代码划分为多个代码块，按需加载
- Tree Shaking：配置Webpack的Tree Shaking机制，去除未使用的代码
- 压缩代码：使用工具如UglifyJS或Terser来压缩JavaScript代码
- 使用生产模式：在Webpack中使用生产模式，通过设置mode: 'production'来启用优化
- 使用压缩工具：使用现代的压缩工具，如Brotli和Gzip，来对静态资源进行压缩
- 利用CDN加速：将项目中引用的静态资源路径修改为CDN上的路径，减少图片、字体等静态资源等打包

4. vite比webpack快在哪里

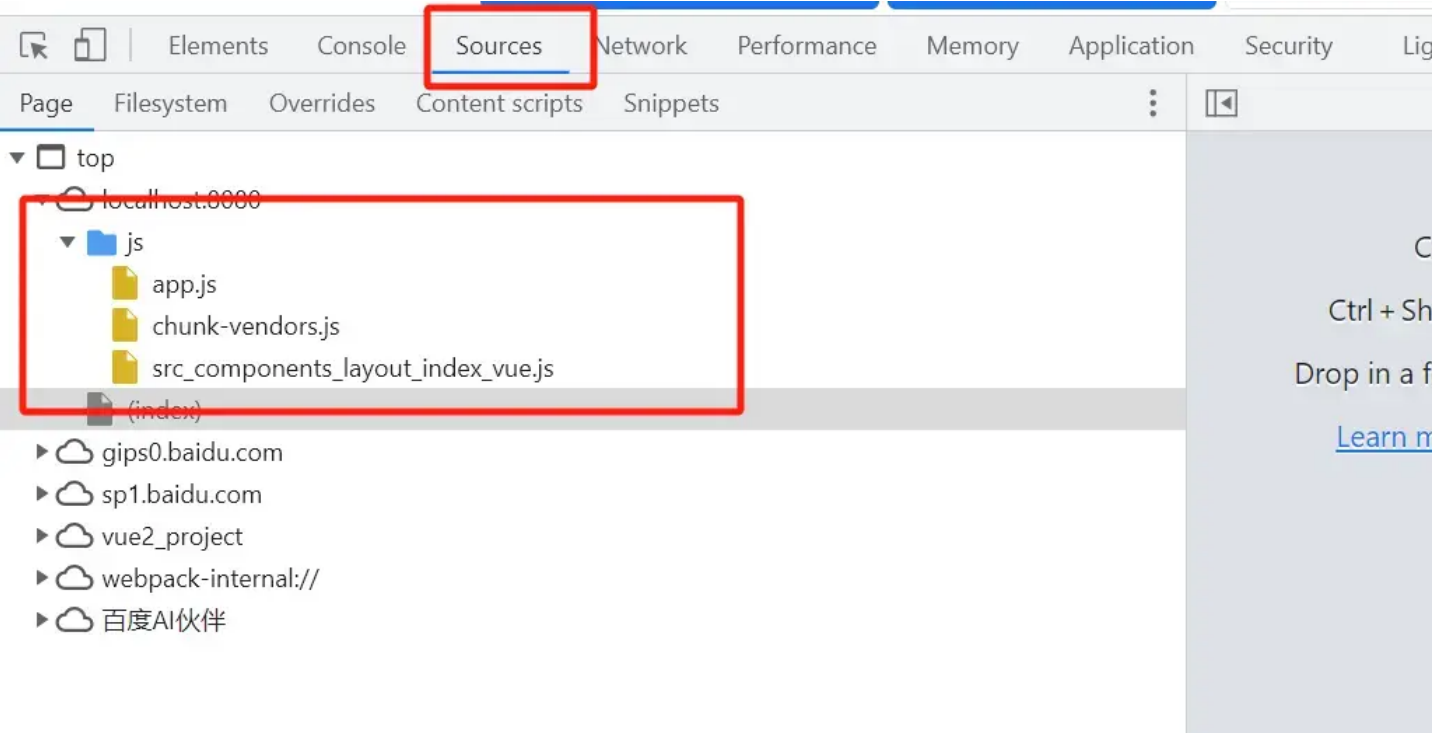
(一)、开发模式的差异

在开发环境中，Webpack 是先打包再启动开发服务器，而 Vite 则是直接启动，然后再按需编译依赖文件。（大家可以启动项目后检查源码 Sources 那里看到）

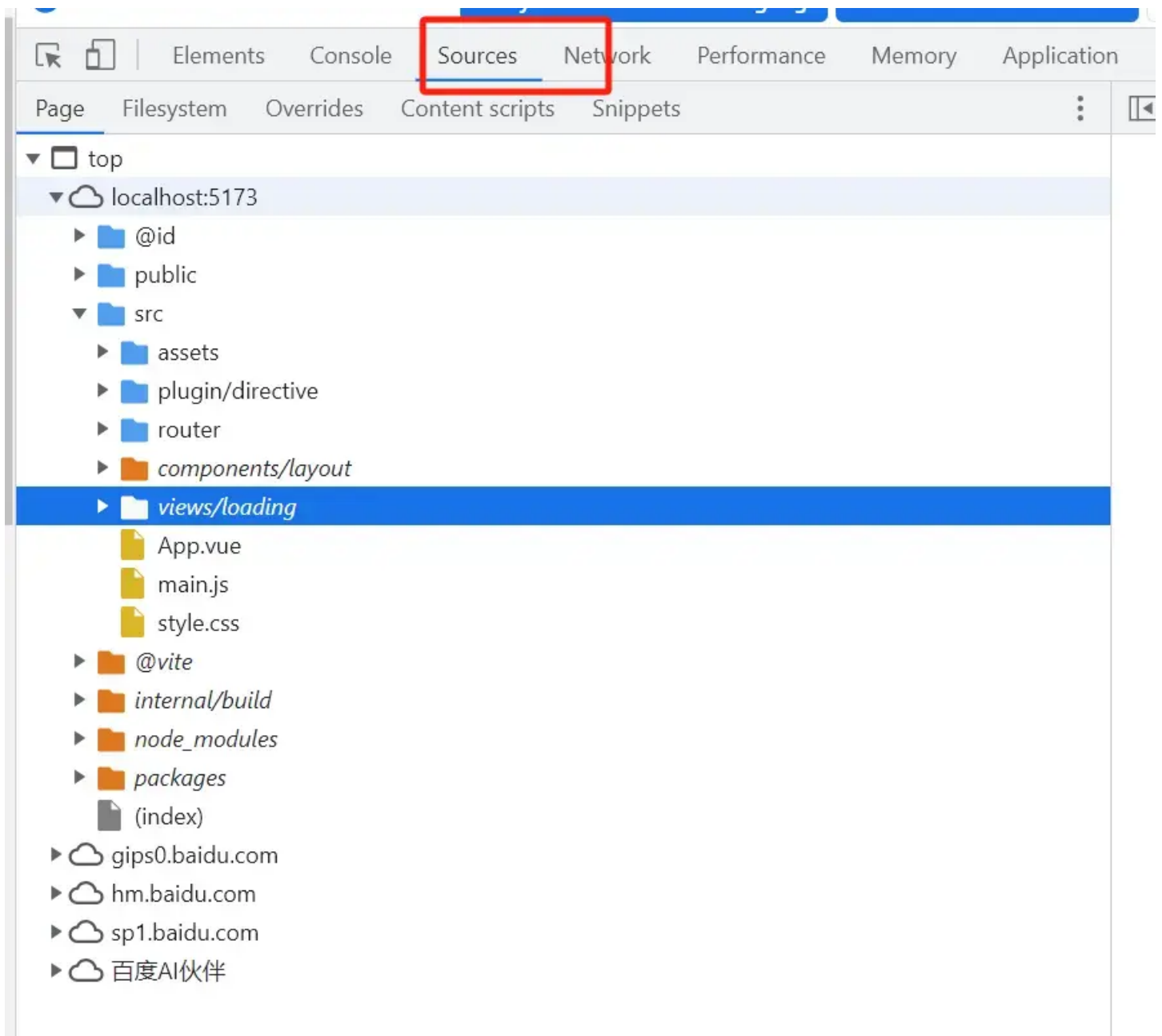
这意味着，当使用 Webpack 时，所有的模块都需要在开发前进行打包，这会增加启动时间和构建时间。

而 Vite 则采用了不同的策略，它会在请求模块时再进行实时编译，这种按需动态编译的模式极大地缩短了编译时间，特别是在大型项目中，文件数量众多，Vite 的优势更为明显。

Webpack启动



Vite启动



（二）、对ES Modules的支持

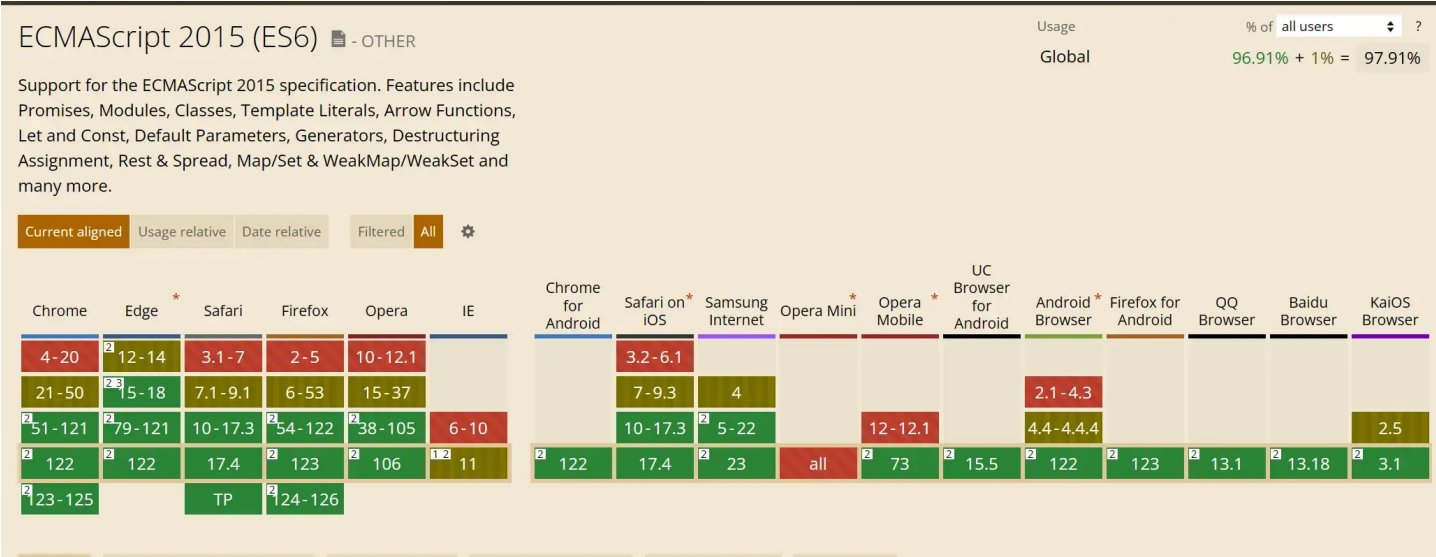
现代浏览器本身就支持 `ES Modules`，会 `主动发起` 请求去获取所需文件。Vite充分利用了这一点，将开发环境下的模块文件直接作为浏览器要执行的文件，而不是像 Webpack 那样 `先打包`，再交给浏览器执行。这种方式减少了中间环节，提高了效率。

什么是ES Modules?

通过使用 `export` 和 `import` 语句，ES Modules 允许在浏览器端导入和导出模块。

当使用 ES Modules 进行开发时，开发者实际上是在构建一个 `依赖关系图`，不同依赖项之间通过导入语句进行关联。

主流浏览器（除IE外）均支持ES Modules，并且可以通过在 `script` 标签中设置 `type="module"` 来加载模块。默认情况下，模块会延迟加载，执行时机在文档解析之后，触发DOMContentLoaded事件前。



(3)、底层语言的差异

Webpack 是基于 Node.js 构建的，而 Vite 则是基于 esbuild 进行预构建依赖。esbuild 是采用 Go 语言编写的，Go 语言是纳秒级别的，而 Node.js 是毫秒级别的。因此，Vite 在打包速度上相比Webpack 有 10-100 倍的提升。

什么是预构建依赖？

预构建依赖通常指的是在项目启动或构建之前，对项目中所需的依赖项进行预先的处理或构建。这样做的好处在于，当项目实际运行时，可以直接使用这些已经预构建好的依赖，而无需再进行实时的编译或构建，从而提高了应用程序的运行速度和效率。

(4)、热更新的处理

在 Webpack 中，当一个模块或其依赖的模块内容改变时，需要重新编译这些模块。而在 Vite 中，当某个模块内容改变时，只需要让浏览器重新请求该模块即可，这大大减少了热更新的时间。

5. 说一下你对Monorepo的理解

Monorepo 是一种项目代码管理方式，指单个仓库中管理多个项目，有助于简化代码共享、版本控制、构建和部署等方面的复杂性，并提供更好的可重用性和协作性。Monorepo 提倡了开放、透明、共享的组织文化，这种方法已经被很多大型公司广泛使用，如 Google、Facebook 和 Microsoft 等。

Monorepo优劣：

场景	MultiRepo	MonoRepo
代码可见性	✅ 代码隔离，研发者只需关注自己负责的仓库 ❌ 包管理按照各自 owner划分，出现问题	✅ 一个仓库中多个相关项目，很容易看到整个代码库的变化趋势，更好的团队协作

	当出现问题时，需要到依赖包中进行判断并解决。	作。❌增加了非owner改动代码的风险
依赖管理	❌多个仓库都有自己的node_modules，存在依赖重复安装情况，占用磁盘内存大。	✅多项目代码都在一个仓库中，相同版本依赖提升到顶层只安装一次，节省磁盘内存，
代码权限	✅各项目单独仓库，不会出现代码被误改的情况，单个项目出现问题不会影响其他项目。	❌多个项目代码都在一个仓库中，没有项目粒度的权限管控，一个项目出问题，可能影响所有项目。
开发迭代	✅仓库体积小，模块划分清晰，可维护性强。 ❌多仓库来回切换（编辑器及命令行），项目多的话效率很低。多仓库见存在依赖时，需要手动 npm link，操作繁琐。❌依赖管理不便，多个依赖可能在多个仓库中存在不同版本，重复安装，npm link 时不同项目的依赖会存在	✅多个项目都在一个仓库中，可看到相关项目全貌，编码非常方便。 ✅代码复用高，方便进行代码重构。 ❌多项目在一个仓库中，代码体积多大几个 G，git clone 时间较长。 ✅依赖调试方便，依赖包迭代场景下，借助工具自动 npm link，直接使用最新版本依赖，简化了

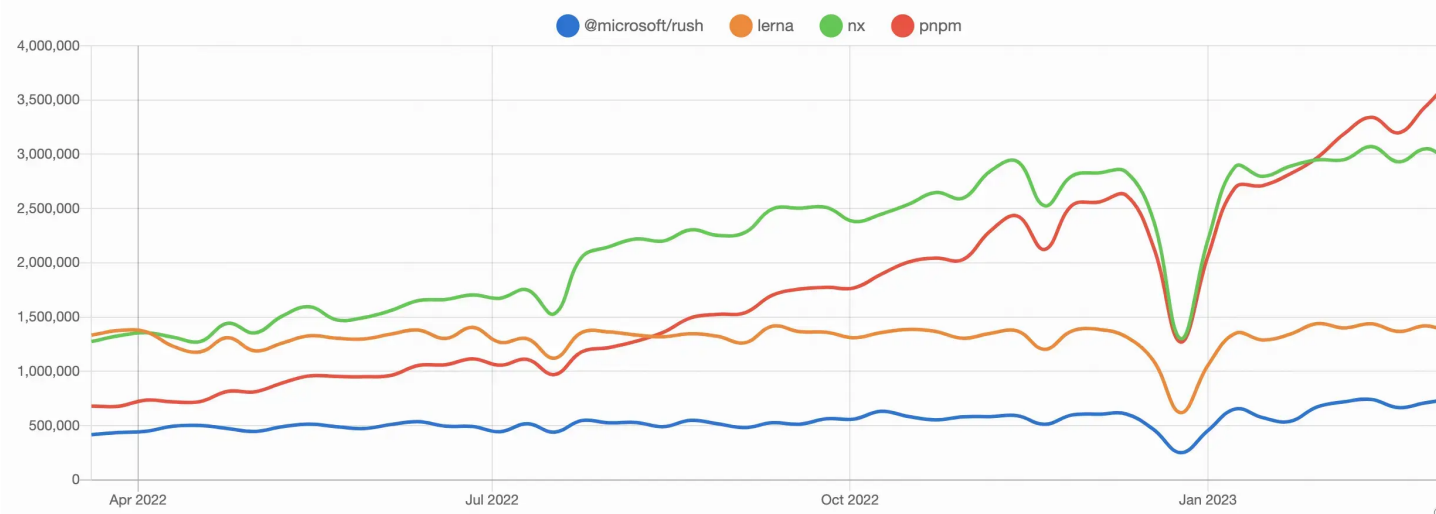
	冲突。	操作流程。
工程配置	 各项目构建、打包、代码校验都各自维护，不一致时会导致代码差异或构建差异。	 多项目在一个仓库，工程配置一致，代码质量标准及风格也很容易一致。
构建部署	 多个项目间存在依赖，部署时需要手动到不同的仓库根据先后顺序去修改版本及进行部署，操作繁琐效率低。	 构建性 Monorepo 工具可以配置依赖项目的构建优先级，可以实现一次命令完成所有的部署。

13.你在项目是怎么做Monorepo?

工具对比

工具	Turborepo	Rush	Nx	Lerna	Pnpm Workspace
依赖管理					
版本管理					
增量构建					
插件扩展					
云端缓存					
Stars	20.4K	4.9K	17K	34.3K	22.7K

Downloads in past 1 Year ▾



详细对比：

- [Nx and Turborepo](#)
- [lerna-vs-turbopack-rush](#)

选型建议

- 建议采用渐进式架构方案，即对于轻量级 Monorepo 项目，我们初期可以选择 Lerna + pnpm workspace + lerna-changelog，解决了依赖管理、发版管理等问题，为开发者带来便利；随着后续项目迭代，代码变多或多个项目间依赖关系复杂，可以很平滑的接入 Nx 来提升构建打包效率。

14.为什么pnpm比npm快

Pnpm 比 npm 快的原因在于其优化的文件存储方式、依赖管理方式以及并行下载能力。 以下是详细介绍：

- Pnpm 使用基于内容寻址的文件系统来存储磁盘上的所有文件，这意味着它不会在磁盘中重复存储相同的依赖包，即使这些依赖包被不同的项目所依赖。这种存储方式使得Pnpm在安装依赖时能够更高效地利用磁盘空间，同时也减少了下载和安装的时间。
- Pnpm 在下载和安装依赖时采用了并行下载的能力，这进一步提高了安装速度。
- Pnpm 还具有一些其他特性，例如节省空间的硬链接和符号链接的使用，这些都有助于提高其性能。

15.ESLint概念及原理

Lint会对代码做静态分析，检查出其中的一些结构错误或者格式错误。在前端领域中，我们常用的lint就是ESLint，它用于检查JavaScript代码是否符合规则。

基本原理

ESLint基本架构图如下：

- `lib/linter/` 这个模块是核心的 `Linter` 类，根据配置选项进行代码验证、检查并修复问题。这个文件不做任何文件 I/O，并且完全不与 `console` 互动。

Linter 是 eslint 最核心的类了，它提供了这几个 api：

- 检查：verify
- 检查并修复：verifyAndFix
- 获取 AST：getSourceCode
- 定义 Parser：defineParser
- 定义 Rule：defineRule
- 获取所有的 Rule：getRules

其中SourceCode指的是AST（抽象语法树），源代码字符串通过Parser解析成AST，之后ESLint就可以通过AST提供的信息与Rules对比，从而给出代码规范分析的结果，指出错误，并且还可以自动修复。

Linter 主要的功能是在 verify 和 verifyAndFix 里实现的，当命令行指定 `--fix` 或者配置文件指定 `fix: true` 就会调用 verifyAndFix 对代码进行检查并修复，否则会调用 verify 来进行检查。

AST

ESLint拿到源代码后会进行parse操作，生成AST用于静态分析。ESLint使用的是Espreesser parser。

Estree是一套AST标准，Esprima基于estree标准实现了AST。Acorn，它在Esprima之后出现，也是estree 标准的实现，但是它速度比 esprima 快，而且支持插件，可以通过插件扩展语法支持。

Espreess最初Fork自Esprima，因为Acorn的各种优点现在它建立在Acorn之上。

下面简单介绍下Espreess解析器下AST的几个常见的节点，也可以在estree中查看更多详情。

Literal

Literal 是字面量的意思，它的值可以是布尔、数字、字符串等。

```

1 let a = 1;
2 let b = 'b';
3 let c = true;

```

Tree

JSON

5ms

- ☒ Autofocus ☒ Hide methods
- ☐ Hide empty keys ☐ Hide location data
- ☐ Hide type keys

```

- declarations: [
  - VariableDeclarator {
    type: "VariableDeclarator"
    start: 28
    end: 36
    + range: [2 elements]
    + id: Identifier {type, start, end, name}
    - init: Literal {
      type: "Literal"
      start: 32
      end: 36
      + range: [2 elements]
      value: true
      raw: "true"
    }
  }
]

```

Identifier

Identifier 是标识符的意思，变量名、属性名、参数名等各种声明和引用的名字，都是Identifier。

```

1 let identifier = 1

```

Tree

JSON

2ms

```

start: 4
end: 18
+ range: [2 elements]
+ id: Identifier {type, start, end, range, name}
- init: Literal {
  type: "Literal"
  start: 17
  end: 18
  + range: [2 elements]
  value: 1
  raw: "1"
}

```

statement

statement 是语句，它是可以独立执行的单位，比如 break、continue、debugger、return 或者 if 语句、while 语句、for 语句，还有声明语句，表达式语句等。我们写的每一条可以独立执行的代码都是语句。语句末尾一般会加一个分号分隔，或者用换行分隔。

下面这些我们经常写的代码，每一行都是一个 Statement：

```
1 javascript
2 复制代码
3 js复制代码break;continue;return;debugger;throw Error();{}try {} catch(e) {}
  finally{}for (let key in obj) {}for (let i = 0;i < 10;i ++) {}while (true) {}do
  {} while (true)switch (v){case 1: break;default:;}with (a){}
```

1 return|

TreeJSON3ms

☒ Autofocus☒ Hide methods☐ Hide empty keys☐ Hide location data☐ Hide type keys

- Program {
 type: "Program"
 start: 0
 end: 6
 + range: [2 elements]
 - body: [
 - ReturnStatement {
 type: "ReturnStatement"
 start: 0
 end: 6
 + range: [2 elements]
 argument: null
 }
]
 sourceType: "module"

Verify & Fix

PreProcess阶段

1.确定是否需要process

```

_verifyWithConfigArray(textOrSourceCode, configArray, options) {
  debug("With ConfigArray: %s", options.filename);

  // Store the config array in order to get plugin envs and rules later.
  internalSlotsMap.get(key: this).lastConfigArray = configArray;

  // Extract the final config for this file.
  const config = configArray.extractConfig(options.filename);
  const processor =
    config.processor && Toru Nagashima, 4 years ago • New: multiple processors supp
    configArray.pluginProcessors.get(config.processor);

  // Verify.
  if (processor) {
    debug("Apply the processor: %o", config.processor);
    const { preprocess, postprocess, supportsAutofix } = processor;
    const disableFixes = options.disableFixes || !supportsAutofix;

    return this._verifyWithProcessor(
      textOrSourceCode,
      config,
      options: { ...options, disableFixes, postprocess, preprocess },
      configForRecursive: configArray
    );
  }

  return this._verifyWithoutProcessors(textOrSourceCode, providedConfig: config,
    providedOptions: options);
}

```

上面介绍过，ESLint 处理器可以从其他类型的文件中提取 JavaScript 代码，然后让 ESLint 对 JavaScript 代码进行检查，这就是processor的作用之一。例如，对vue类型文件做ESLint检查，processor就派上用场了。更详细的介绍可以看我另外一篇文章 [Processor](#)

Parse阶段

1.确定parser:

默认是 ESLint 自带的Espree，也可以通过配置来切换成别的 parser，比如 @eslint/babel-parser、@typescript/eslint-parser 等。


```
enabledEnvs: undefined
envInFile: undefined
languageOptions: undefined
lintingProblems: undefined
> options: {filename: '/Users/krialy/Desktop/...',
> parser: {Syntax: {...}, VisitorKeys: {...}, ...
  parserName: 'espreet'
  parserOptions: undefined
> providedConfig: ExtractedConfig {configN...
> providedOptions: {allowInlineConfig: tru...
  resolvedEnvConfig: undefined
  settings: undefined
> slots: {cwd: '/Users/krialy/Desktop/esli...
CALL STACK
BREAKPOINTS
  Caught Exceptions
  Uncaught Exceptions
  eslint.js source-code/eslint/bin 120
  eslint.js source-code/eslint/bin 128
  linter.js source-code/eslint/lib/li... 1299:13
  linter.js source-code/eslint/lib/li... 1333:35
  linter.js source-code/es... 1333:35

1310 // Resolve parser.
1311 let parserName = DEFAULT_PARSER_NAME;
1312 let parser = espreet;
1313
1314 if (typeof config.parser === "object" && config.parser !== null) {
1315   parserName = config.parser.filePath;
1316   parser = config.parser.definition;
1317 } else if (typeof config.parser === "string") {
1318   if (!slots.parserMap.has(key: config.parser)) {
1319     return [
1320       ruleId: null,
1321       fatal: true,
1322       severity: 2,
1323       message: `Configured parser '${config.parser}' was not found.`,
1324       line: 0,
1325       column: 0
1326     ];
1327   }
1328   parserName = config.parser;
1329   parser = slots.parserMap.get(key: config.parser);
```

2. 执行parse生成Source Code (AST) :

```
source-code > eslint > lib > linter > JS linter.js > Linter > _verifyWithoutProcessors

1345 const languageOptions = createLanguageOptions({
1346   globals: config.globals,
1347   parser,
1348   parserOptions
1349 });
1350
1351 if (!slots.lastSourceCode) {
1352   const parseResult = parse(
1353     text,
1354     languageOptions,
1355     options.filename
1356   );
1357
1358   if (!parseResult.success) {
1359     return [parseResult.error];
1360   }
1361 }
```

VARIABLES

- Block: _verifyWithoutProcessors
 - parseResult: {success: true, sourceCode: ...
 - sourceCode: SourceCode {hasBOM: false, ...
 - _commentCache: WeakMap
 - ast: Node {type: 'Program', start: 0, ...
 - hasBOM: false
 - lines: (1) ['let a = 1']
 - lineStartIndices: (1) [0]
 - parserServices: {}
 - scopeManager: ScopeManager {scopes: Ar...
 - Symbol(comments): (0) []
 - Symbol(indexMap): {0: 0, 2: 0, 4: 1, 6...
 - Symbol(tokens): (4) [Token, Token, Tok...
 - text: 'let a = 1'
 - tokensAndComments: (4) [Token, Token, ...
 - visitorKeys: {AssignmentExpression: Ar...
 - [[Prototype]]: TokenStore

检查阶段

调用rule对SourceCode检查，生成linting problems

```
Local
  ruleId: 'no-class-assign'
  this: undefined
> Closure (_verifyWithoutProcessors)
> Closure
> Global

1394 let lintingProblems;
1395
1396 try {
1397   lintingProblems = runRules(
1398     sourceCode,
1399     configuredRules,
1400     ruleId => getRule(slots, ruleId),
1401     parserName,
1402     languageOptions,
1403     settings,
1404     options.filename,
1405     options.disableFixes,
1406     slots.cwd,
1407     providedOptions.physicalFilename
```

那么是如何检查？

1. 遍历AST并存储AST Node

```
const nodeQueue = [];  
let currentNode = sourceCode.ast;  
  
Traverser.traverse(node: sourceCode.ast, options: {  
  enter(node: any, parent: any) {  
    node.parent = parent;  
    nodeQueue.push(...items: { isEntering: true, node });  
  },  
  leave(node: any) {  
    nodeQueue.push(...items: { isEntering: false, node });  
  },  
  visitorKeys: sourceCode.visitorKeys  
});
```

2. 遍历规则列表

The screenshot shows the VS Code editor with the ESLint configuration file. On the left, the 'configuredRules' section is expanded, showing a list of rules with their severity levels. The rule 'constructor-super' is highlighted. On the right, the ESLint source code is visible, showing the logic for iterating over the configured rules. The code uses 'Object.keys' to get the keys of 'configuredRules' and 'forEach' to iterate over them. The 'severity' of each rule is checked, and if it is not 0, the rule is mapped and its listeners are created.

为每条规则添加对应AST Node的Listener

The screenshot shows the VS Code editor with the ESLint source code. The 'runRules' function is expanded, showing the logic for iterating over the configured rules. The 'ruleListeners' are created for each rule, and the 'createRuleListeners' function is called to bind the listeners to the AST nodes. The 'ruleListeners' are stored in the 'ruleContext' object.

为constructor-super规则绑定对应Listener (ReturnStatement、"Program:exit"等)，当AST遍历执行到ReturnStatement类型的节点的时候便会执行constructor-super规则ReturnStatement方法里的逻辑。

```
Local: create
> context: {id: 'constructor-super', options: Array(0), report: ...
  funcInfo: undefined
> isCalledInEveryPath: f isCalledInEveryPath(segment) {
> isCalledInSomePath: f isCalledInSomePath(segment) {
  segInfoMap: undefined
> this: Object
> Closure
> Global

CALL STACK
Node.js Process: eslint.js [20418] PAUSED
  create source-code/eslint/lib/rules/constructor-super.js 153:24
  createRuleListeners source-code/eslint/lib/linter/linter.js
  <anonymous> source-code/eslint/lib/linter/linter.js 1108:110
  runRules source-code/eslint/lib/linter/linter.js 1045:34

BREAKPOINTS
WATCH

290
291
292
293 > onCodePathSegmentLoop(fromSegment, toSegment) {
328
329
330
331 /**
332  * Checks for a call of `super()`.
333  * @param {ASTNode} node A CallExpression node to check.
334  * @returns {void}
335  */
336 "CallExpression:exit"(node) {
383
384
385
386 /**
387  * Set the mark to the returned path as `super()` was called.
388  * @param {ASTNode} node A ReturnStatement node to check.
389  * @returns {void}
390  */
391 ReturnStatement(node) {
412
413
414
415 /**
416  * Resets state.
417  * @returns {void}
418  */
419 "Program:exit"() {
420
```

3.Emit对应AST Node的Listener

```
CALL STACK
Node.js Process: eslint.js [181...] PAUSED
  applySelector source-code/eslint/lib/li...
  applySelectors source-code/eslint/lib/li...
  enterNode source-code/eslint/lib/linter/...
  enterNode source-code/eslint/lib/linter/...
  <anonymous> source-code/eslint/lib...
  runRules source-code/eslint/lib/linter/li...
  _verifyWithoutProcessors source-cod...
  _verifyWithConfigArray source-code/...
  verify source-code/eslint/lib/linter/lint...
  verifyAndFix source-code/eslint/lib/lin...
  verifyText source-code/eslint/lib/cli-e...

1148
1149
1150 nodeQueue.forEach(callbackfn: traversalInfo : any => {
1151   currentNode = traversalInfo.node;
1152   try {
1153     if (traversalInfo.isEntering) {
1154       eventGenerator.enterNode(node: currentNode);
1155     } else {
1156       eventGenerator.leaveNode(node: currentNode);
1157     }
1158   } catch (err) {
1159     err.currentNode = currentNode;
1160     throw err;
1161   }
1162 }

Local: applySelector
> node: Node {type: 'VariableDeclaration',...
> selector: {rawSelector: 'VariableDeclar...
  attributeCount: 0
  identifierCount: 1
  isExit: false
> listenerTypes: (1) ['VariableDeclaratio...
> parsedSelector: {type: 'identifier', va...
  rawSelector: 'VariableDeclaration'
> [[Prototype]]: Object
> this: NodeEventGenerator
> Closure

288
289
290 /**
291  * Checks a selector against a node, and emits it if it matches
292  * @param {ASTNode} node The node to check
293  * @param {ASTSelector} selector An AST selector descriptor
294  * @returns {void}
295  */
296 applySelector(node, selector) {
297   if (esquery.matches(t: node, r: selector.parsedSelector, n: this.currentAncestry,
298     o: this.esqueryOptions)) {
299     this.emitter.emit(selector.rawSelector, node);
300   }
301 }
```

这样AST Node遍历完成后也就执行所有的rules了。

在执行rules的过程中对比AST发现和rule规则不匹配，就可以添加问题到linting problems


```

const ruleContext = Object.freeze(
  f: Object.assign(
    target: Object.create(o: sharedTraversalContext),
    source: {
      id: ruleId,
      options: getRuleOptions(ruleConfig: configuredRules[ruleId]),
      report(...args: any[]) {
        if (reportTranslator === null) {
          ...
        }
        const problem = reportTranslator(...args);

        if (problem.fix && !(rule.meta && rule.meta.fixable)) {
          ...
        }
        if (problem.suggestions && !(rule.meta && rule.meta.hasSuggestions === true)) {
          ...
        }
        lintingProblems.push(...items: problem);
      }
    }
  );
);

const ruleListeners = timing.enabled ? timing.time(key: ruleId,
fn: createRuleListeners)(rule, ruleContext) : createRuleListeners(rule, ruleContext);

```

VARIABLES

Local

> addRuleErrorHandler: f addRuleErrorHandl...

> messageIds: {missingSome: 'Lacked a call...

reportTranslator: null

> rule: {meta: {...}, create: f}

> ruleContext: {id: 'constructor-super', o...

ruleId: 'constructor-super'

> ruleListeners: {onCodePathStart: f, onCo...

severity: 2

this: undefined

> Closure (runRules)

> Closure

CALL STACK

Node.js Proces... PAUSED ON BREAKPOINT

<anonymous> source-code/eslint/lib/lint...

runRules source-code/eslint/lib/linter/li...

_verifyWithoutProcessors source-cod...

_verifyWithConfigArray source-code/...

BREAKPOINTS

WATCH

source-code > eslint > lib > rules > JS constructor-super.js > <unknown> > create > "CallExpression:exit"

> report

duplicate = duplicate || info.calledInSomePaths;

info.calledInSomePaths = info.calledInEveryPaths =

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

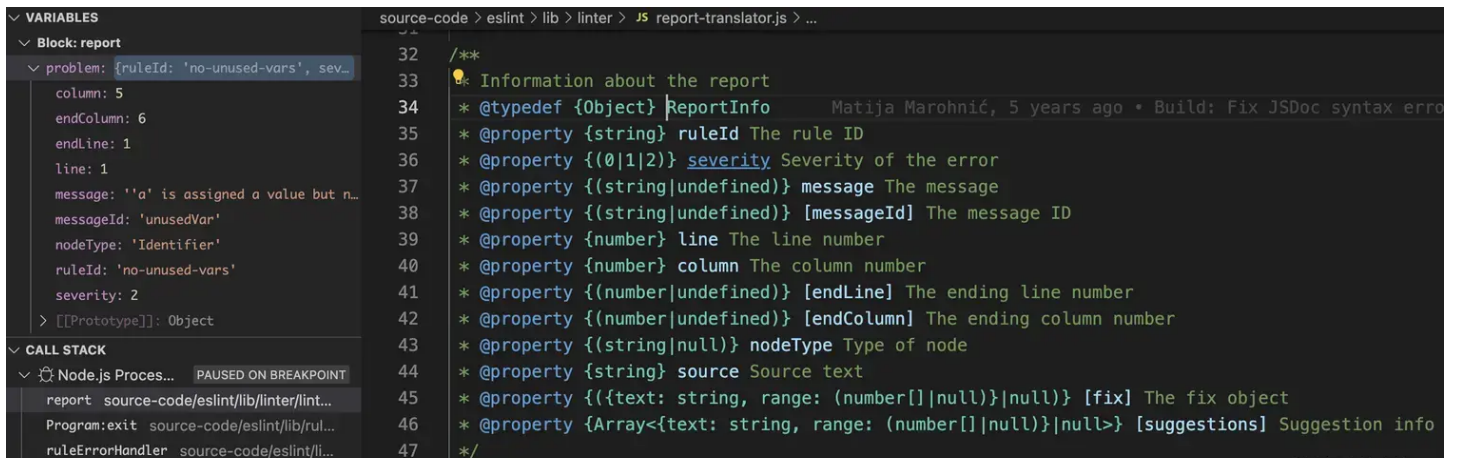
```

    }
  }

  if (info) {
    if (duplicate) {
      context.report({
        messageId: "duplicate",
        node
      });
    } else if (!funcInfo.superIsConstructor) {
      context.report({
        messageId: "badSuper",
        node
      });
    } else {
      info.validNodes.push(node);
    }
  }

```

最后生成的linting problems就是lint检查结果了。从哪一行（line）哪一列（column）到哪一行（endLine）哪一列（endColumn），有什么错误（message）。



问：在检查阶段为什么需要先存储AST Node然后再从AST Node Queue遍历来Emit Listener呢？这样不是遍历两次了吗？

因为rules一直有个小问题，node的parent属性只会在节点被遍历后才能被访问到。为了解决这个问题ESLint延迟执行了Emit，这样node parent属性就可以被访问到了。相关issue：

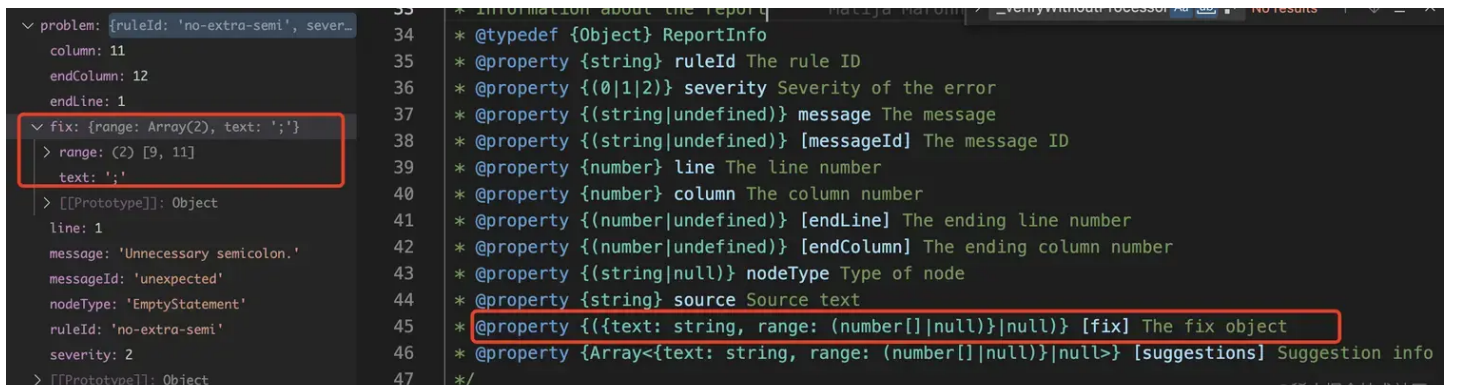
github.com/eslint/esli...

PostProcess阶段

这个阶段主要用来对生成的linting problems做一些处理，例如过滤、修改之类的。

Fix阶段

对于可以fix的规则在lint检查完后会，linting problems里会有生成的fix信息，用于自动修复问题。



其中range表示范围，text表示替换的内容。结合到一起就是，range内的字符串替换成text即完成修复。

举个例子：

1.源代码

2.配置rule：no-extra-semi，不允许多余的分号。 3.运行ESLint检查，生成如下结构

```

    problem: {ruleId: 'no-extra-semi', severity: 2,
      column: 11,
      endColumn: 12,
      endLine: 1,
      fix: {range: Array(2), text: ';' }
    }
  ],
  messages: [
    {
      > range: (2) [9, 11]
      text: ';'
      > [[Prototype]]: Object
      line: 1
      message: 'Unnecessary semicolon.'
      messageId: 'unexpected'
      nodeType: 'EmptyStatement'
      ruleId: 'no-extra-semi'
      severity: 2
    }
  ]
}

```

```

1  css
2  复制代码
3  js复制代码{    fix: {      range: [9, 11],      text: ';'    }}

```

表示替换源码字符串（中index从9到11的内容为';'，即替换";;"为";"，替换后结果如下：

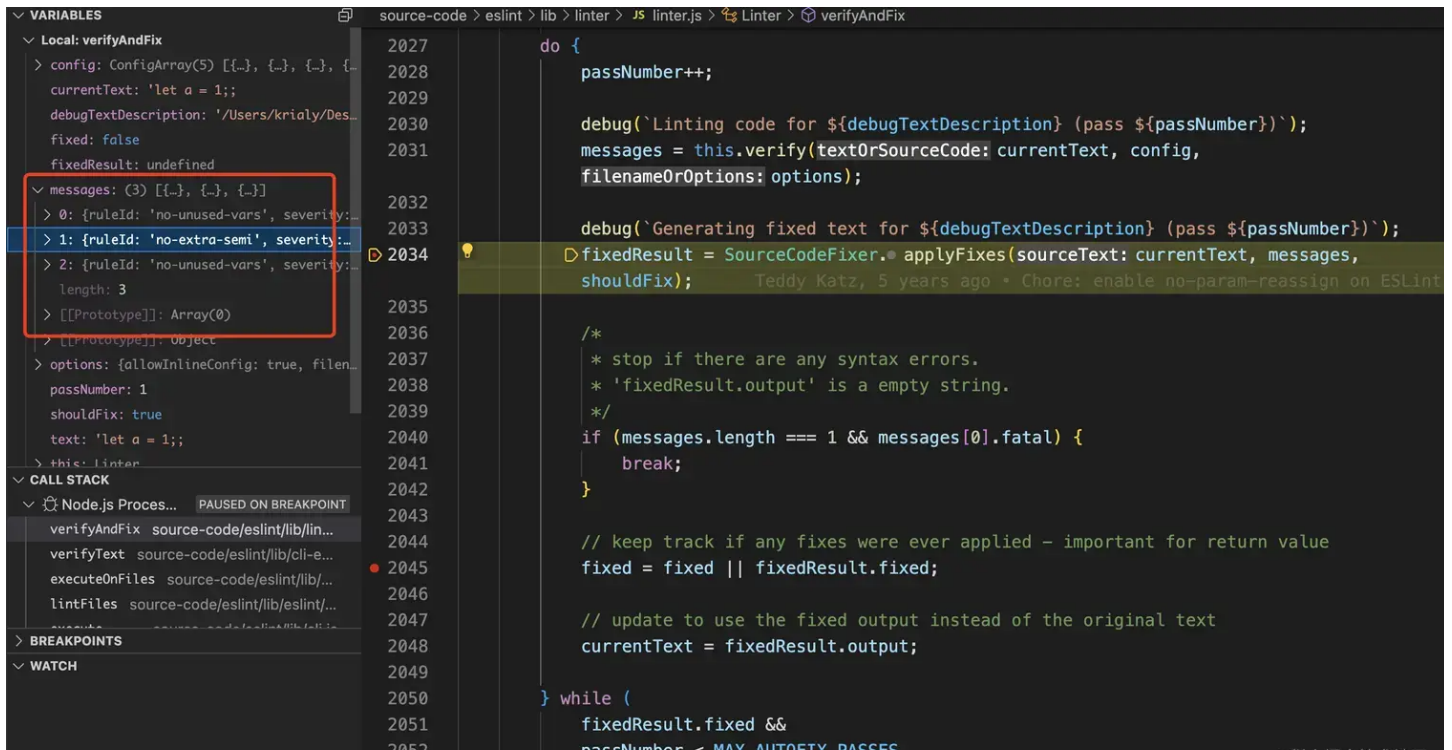
```

1  let a = 1;
2
3

```

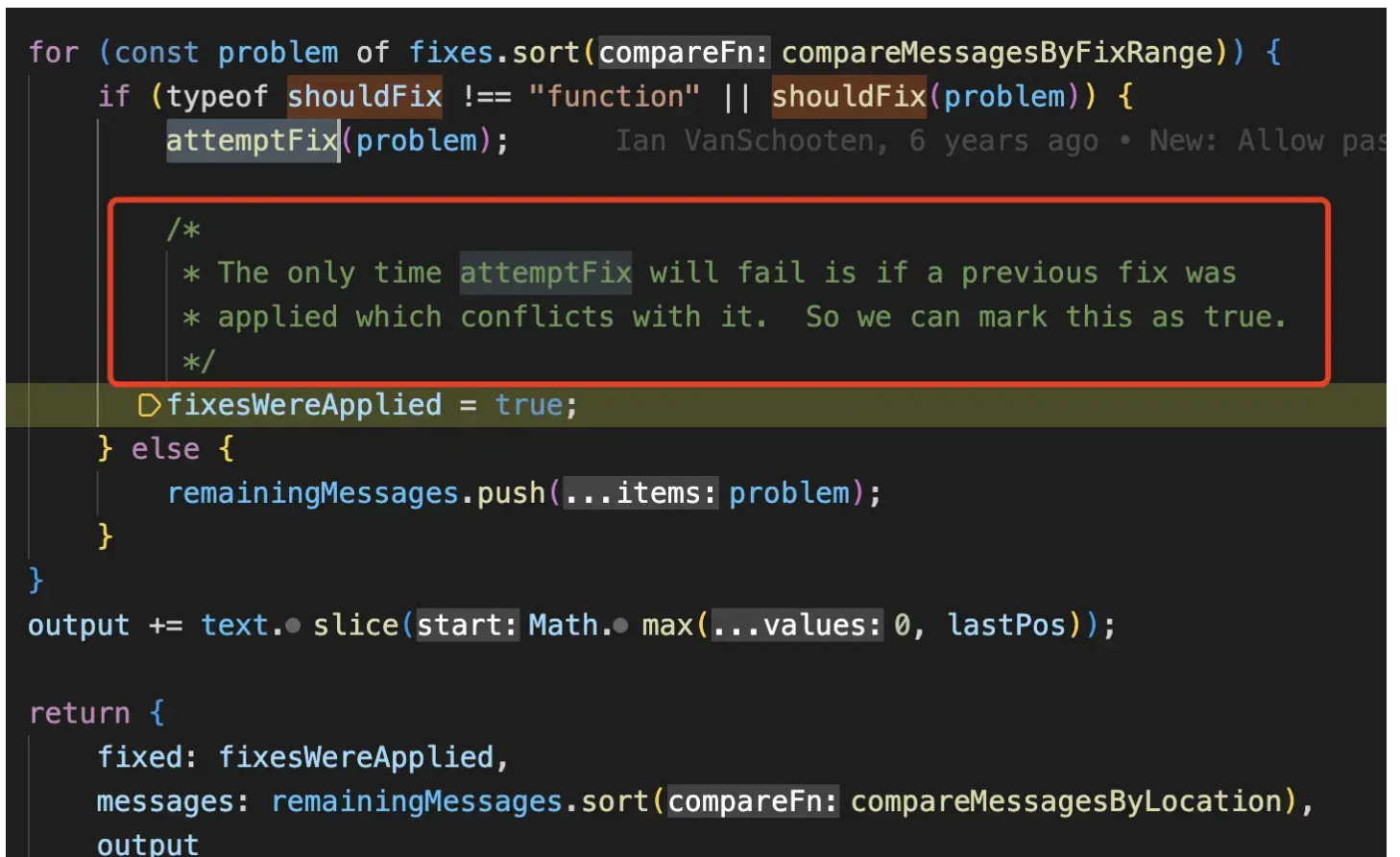
源码fix流程分析：

1.根据linting problems替换



问：这里为什么要加循环呢？

答：因为多个linting problem之间的range也就是替换的范围可能是有重叠的，如果有重叠就放到下一次来修复，下一次修复则会根据当前修复过一次的代码再继续verify，生成linting problems，以此循环直至没有problem可以修复。不过这样的循环最多修复 10 次，如果还有linting problems没修复就不修了。



2. 替换算法：其实就是简单的字符串拼接与替换。

至此，ESLint工作的主要流程完成。

16.Bable概念及原理

Babel 是一个流行的用于将新版本 ES6+代码转换为向后兼容版本（ES5）代码的JavaScript编译器。它还为 JSX语法提供了编译 支持，还有一些其他插件可用于转换特定类型的代码。

- 1 复制代码
- 2 Babel 的工作原理：三类功能

解析

当 Babel 接收到源代码时，将会调用一个叫做解析器的工具，用于将源代码转换为抽象语法树（AST）。在这个过程中，解析器会识别代码中的语法结构，并将其转换为对应的节点类型。例如，当解析器遇到一个变量声明语句时，它将会创建一个“VariableDeclaration”节点，并将该节点的信息存储在 AST 中。AST 是一个以节点为基础组成的树形结构，每个节点都有相应的类型、属性和子节点等信息。

转换

一旦 AST 被创建，Babel 将遍历整个树形结构，对每个节点进行转换。这些转换可以是插件、预设或手动创建的。转换器会检查 AST 中的每个节点，然后对其进行相应的修改或替换，以将新语法转换为旧语法。例如，如果 Babel 遇到一个包含箭头函数的节点，而你已经启用了转换插件，该插件将会将箭头函数转换为其等效的体函数。代码转换后，Babel 将会生成一个新的 AST。

生成

最后，Babel 将基于转换后的 AST 生成代码文本。在这个步骤中，Babel 将遍历转换后的 AST，并创建对应的代码字符串，并将这些字符串组合成一个完整的 JavaScript 文件。如果启用了代码压缩，Babel 还可以将生成的代码进行压缩。总结来说，Babel 的原理就是将 JavaScript 源代码转换为抽象语法树（AST），然后对 AST 进行转换，生成与源代码功能相同但向后兼容的代码。Babel 提供了一个强大的生态系统，使得开发者可以轻松扩展并自定义转换器，实现自己的功能需求。

17.npm install 的执行过程

npm install 是 Node.js 包管理器 (npm) 的一个命令，用于安装一个项目所依赖的模块。

执行过程大致如下：

1. 读取 package.json 文件，该文件列出了项目所需要的依赖。
2. 根据 package.json 中的依赖信息以及 node_modules 目录状态，npm 会决定哪些模块需要下载和安装。
3. npm 会查看每个模块的可用版本，并选择符合 package.json 中指定版本范围的最新版本进行安装。

4. 下载所需模块到本地的 `node_modules` 目录。
5. 如果模块包含子模块（`package.json` 中 `dependencies` 或 `devDependencies` 中的模块），则递归执行上述步骤安装这些子模块。

18.npm run start 的整个过程？

`npm run start` 是一个常见的命令，用于启动基于 Node.js 的应用程序。这个命令实际上是一个快捷方式，它告诉 npm 运行在 `package.json` 文件中定义的 "start" 脚本。

当你执行 `npm run start` 时，以下是发生的事情：

1. 查找当前目录下的 `package.json` 文件。
2. 在 `package.json` 文件中，找到 "scripts" 对象。
3. 在 "scripts" 对象中，找到 "start" 键。
4. 执行与 "start" 键关联的命令字符串。

例如，如果你的 `package.json` 文件中的 "scripts" 对象像这样：

```
1 json
2 复制代码
3 "scripts": { "start": "node app.js"}
```

当你运行 `npm run start` 时，npm 将执行 `node app.js`。

这是一个简单的例子，实际的 "start" 脚本可能会包含更多步骤，比如预处理、打包、转译、加载模块绑定等。

总结：`npm run start` 执行 `package.json` 中定义的 "start" 脚本，这个脚本可以启动一个 Node.js 应用程序或执行更复杂的前端构建过程。

19.对 CSS 工程化的理解

CSS 工程化是为了解决以下问题：

1. 宏观设计：CSS 代码如何组织、如何拆分、模块结构怎样设计？
2. 编码优化：怎样写出更好的 CSS？
3. 构建：如何处理我的 CSS，才能让它的打包结果最优？
4. 可维护性：代码写完了，如何最小化它后续的变更成本？如何确保任何一个同事都能轻松接手？

以下三个方向都是时下比较流行的、普适性非常好的 CSS 工程化实践：

- 预处理器：Less、Sass 等；
- 重要的工程化插件：PostCss；

- Webpack loader 等。

基于这三个方向，可以衍生出一些具有典型意义的子问题，这里我们逐个来看：

(1) 预处理器：为什么要用预处理器？它的出现是为了解决什么问题？预处理器，其实就是 CSS 世界的“轮子”。预处理器支持我们写一种类似 CSS、但实际并不是 CSS 的语言，然后把它编译成 CSS 代码：



那为什么写 CSS 代码写得好好的，偏偏要转去写“类 CSS”呢？这就和本来用 JS 也可以实现所有功能，但最后却写 React 的 jsx 或者 Vue 的模板语法一样。

随着前端业务复杂度的提高，前端工程中对 CSS 提出了以下的诉求：

- 宏观设计上：我们希望能优化 CSS 文件的目录结构，对现有的 CSS 文件实现复用；
- 编码优化上：我们希望能写出结构清晰、简明易懂的 CSS，需要它具有一目了然的嵌套层级关系，而不是无差别的一铺到底写法；我们希望它具有变量特征、计算能力、循环能力等等更强的可编程性，这样我们可以少写一些无用的代码；
- 可维护性上：更强的可编程性意味着更优质的代码结构，实现复用意味着更简单的目录结构和更强的拓展能力，这两点如果能做到，自然会带来更强的可维护性。

这三点是传统 CSS 所做不到的，也正是预处理器所解决掉的问题。预处理器普遍会具备这样的特性：

- 嵌套代码的能力，通过嵌套来反映不同 css 属性之间的层级关系；
- 支持定义 css 变量；
- 提供计算函数；
- 允许对代码片段进行 extend 和 mixin；
- 支持循环语句的使用；
- 支持将 CSS 文件模块化，实现复用。

(2) PostCss：PostCss 是如何工作的？我们在什么场景下会使用 PostCss？



它和预处理器的不同就在于，预处理器处理的是类CSS，而 PostCss 处理的就是 CSS 本身。Babel 可以将高版本的 JS 代码转换为低版本的 JS 代码。PostCss 做的是类似的事情：它可以编译尚未被浏览器广泛支持的先进的 CSS 语法，还可以自动为一些需要额外兼容的语法增加前缀。更强的是，由于 PostCss 有着强大的插件机制，支持各种各样的扩展，极大地强化了 CSS 的能力。

PostCss 在业务中的使用场景非常多：

- 提高 CSS 代码的可读性：PostCss 其实可以做类似预处理器能做的工作；
- 当我们的 CSS 代码需要适配低版本浏览器时，PostCss 的 Autoprefixer 插件可以帮助我们自动增加浏览器前缀；
- 允许我们编写面向未来的 CSS：PostCss 能够帮助我们编译 CSS next 代码；

(3) Webpack 能处理 CSS 吗？如何实现？

- Webpack 在裸奔的状态下，是不能处理 CSS 的，Webpack 本身是一个面向 JavaScript 且只能处理 JavaScript 代码的模块化打包工具；
- Webpack 在 loader 的辅助下，是可以处理 CSS 的。

如何用 Webpack 实现对 CSS 的处理：

- Webpack 中操作 CSS 需要使用的两个关键的 loader：css-loader 和 style-loader
- 注意，答出“用什么”有时候可能还不够，面试官会怀疑你是不是在背答案，所以你还需要了解每个 loader 都做了什么事情：
- css-loader：导入 CSS 模块，对 CSS 代码进行编译处理；
- style-loader：创建style标签，把 CSS 内容写入标签。